

Signature Hero: Fullscreen GLSL Fragment Shader (FE-AA3)

Project: EcoHive Kenya Ltd.

Author: Eric Munyi

Component: `src/components/AmberShaderHero.tsx`

Live URL: <https://ecohive-jet.vercel.app/>

1. Concept & Visual Signature

The hero section renders a custom WebGL fragment shader representing **"Organic Amber Honey Caustics & Bio-Resin Fluid Dynamics"**.

- Simulates fluid ripples of raw Kenyan acacia honey and recycled HDPE polymer matrix.
 - Interactively reacts to mouse cursor position with fluid damping and soft caustics refraction.
 - Maintained high optical contrast with foreground typography (`#1C1917` deep stone text on `#FDFBF7` warm cream base).
-

2. Responsible Shipping & Performance Fallbacks

One-liner on Reduced-Motion & Perf Fallback:

"Caps `devicePixelRatio` at `2.0`, pauses the animation loop via `document.hidden` when the browser tab is backgrounded, and gracefully falls back to a static ambient gradient if `prefers-reduced-motion: reduce` is enabled or WebGL is unsupported."

- **DPR Capping:** `Math.min(window.devicePixelRatio || 1, 2.0)` avoids thermal throttling and GPU memory spikes on Retina/4K displays.
 - **Visibility Handling:** Listening to `visibilitychange` halts `requestAnimationFrame` when the user leaves the tab, preserving battery and CPU cycles.
 - **Accessibility (`prefers-reduced-motion`):** Evaluates `window.matchMedia('(prefers-reduced-motion: reduce)')`. When active, it renders a single static snapshot without initiating the RAF loop, supplemented by a CSS radial ambient fallback.
-

3. Shader Source Code with Section Commentary

```
precision mediump float;

// Uniforms supplied from the host React/WebGL harness
uniform float u_time;           // Elapsed animation time in seconds
uniform vec2  u_resolution;     // Canvas dimensions in physical pixels (capped at 2x DPR)
uniform vec2  u_mouse;          // Damped mouse cursor coordinates (WebGL coordinate space)
```

```

varying vec2 v_uv;

// =====
// SECTION 1: Simplex Noise & Permutation Tables
// Generates continuous, organic pseudorandom gradients without tiling artifacts.
// =====
vec3 mod289(vec3 x) { return x - floor(x * (1.0 / 289.0)) * 289.0; }
vec2 mod289(vec2 x) { return x - floor(x * (1.0 / 289.0)) * 289.0; }
vec3 permute(vec3 x) { return mod289(((x * 34.0) + 1.0) * x); }

float snoise(vec2 v) {
    const vec4 C = vec4(0.211324865405187, 0.366025403784439, -0.577350269189626,
0.024390243902439);
    vec2 i = floor(v + dot(v, C.yy));
    vec2 x0 = v - i + dot(i, C.xx);
    vec2 i1 = (x0.x > x0.y) ? vec2(1.0, 0.0) : vec2(0.0, 1.0);
    vec4 x12 = x0.xyxy + C.xxzz;
    x12.xy -= i1;
    i = mod289(i);
    vec3 p = permute(permute(i.y + vec3(0.0, i1.y, 1.0)) + i.x + vec3(0.0, i1.x,
1.0));
    vec3 m = max(0.5 - vec3(dot(x0,x0), dot(x12.xy,x12.xy), dot(x12.zw,x12.zw)),
0.0);
    m = m*m;
    m = m*m;
    vec3 x = 2.0 * fract(p * C.www) - 1.0;
    vec3 h = abs(x) - 0.5;
    vec3 ox = floor(x + 0.5);
    vec3 a0 = x - ox;
    m *= 1.79284291400159 - 0.85373472095314 * (a0*a0 + h*h);
    vec3 g;
    g.x = a0.x * x0.x + h.x * x0.y;
    g.yz = a0.yz * x12.xz + h.yz * x12.yw;
    return 130.0 * dot(m, g);
}

// =====
// SECTION 2: Main Fragment Shader Pipeline
// =====
void main() {
    // 1. Normalize fragment coordinates to center origin [-1.0, 1.0] with aspect
    ratio correction
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;
    vec2 p = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / min(u_resolution.x,
u_resolution.y);

    // 2. Interactive Mouse Attractor
    // Calculates distance to cursor and bends fluid flow lines toward user position
    vec2 mouse = (u_mouse * 2.0 - u_resolution.xy) / min(u_resolution.x,
u_resolution.y);
    float mouseDist = length(p - mouse);
    vec2 mousePush = normalize(p - mouse + 0.0001) * (0.22 / (mouseDist * 3.0 +
0.8));

```

```

// 3. Multi-Octave Viscous Fluid Simulation
// Cascades 2 noise layers with slow temporal drift to create viscous honey
waves
float t = u_time * 0.25;
vec2 flowUV = p * 1.25 + mousePush;

float n1 = snoise(flowUV + vec2(t * 0.35, -t * 0.25));
float n2 = snoise(flowUV * 2.0 + vec2(-t * 0.2, t * 0.3) + vec2(n1 * 0.35));

// Optical caustics synthesis (light concentrating through curved honey ripples)
float caustics = sin(n1 * 3.1415 + t) * cos(n2 * 3.1415 - t * 0.6);
caustics = pow(abs(caustics), 2.2);

// 4. Color Palette & Optical Mixing
// High-contrast palette tuned to preserve >7:1 WCAG contrast with dark text
vec3 baseCream = vec3(0.992, 0.984, 0.968); // #FDFBF7 brand background
vec3 lightHoney = vec3(0.996, 0.886, 0.584); // Amber 200
vec3 goldenAmber = vec3(0.961, 0.690, 0.184); // Amber 400
vec3 deepNectar = vec3(0.851, 0.490, 0.114); // Amber 600

vec3 col = baseCream;
float intensity = smoothstep(-0.35, 0.85, n1 + n2 * 0.45 + caustics * 0.4);
col = mix(col, lightHoney, intensity * 0.38);
col = mix(col, goldenAmber, pow(intensity, 2.2) * 0.24);

// Interactive mouse bloom (radiant golden glow under cursor)
float mouseGlow = smoothstep(1.0, 0.0, mouseDist);
col = mix(col, deepNectar, mouseGlow * 0.16 * (0.6 + 0.4 * caustics));

// 5. Film Grain Micro-Texture
// Prevents 8-bit banding on smooth gradients and imparts natural tactile feel
float grain = fract(sin(dot(uv + fract(u_time * 0.005), vec2(12.9898, 78.233)))
* 43758.5453);
col += (grain - 0.5) * 0.016;

gl_FragColor = vec4(col, 1.0);
}

```

4. Mentor Walkthrough Script (Cheat Sheet)

If asked to explain this shader to a mentor or reviewer on camera:

1. The Mental Model (**uv**, **time**, **mouse**):

- "Every pixel calculates its position relative to the center (**p**). **u_time** slowly advances the coordinates of two overlapping simplex noise octaves to produce viscous, organic liquid motion rather than mechanical waves."

2. The Mouse Uniform (**u_mouse**):

- "We calculate the vector between the current pixel and **u_mouse**. The closer the pixel is to the cursor, the more it refracts the fluid flow coordinates towards the cursor via **mousePush**, giving the

impression of stirring warm honey."

3. **Typography & Readability Contrast:**

- "A common pitfall with shaders is muddying readability. Here, the base layer is matched directly to *#FDFBF7*, and the amber values are blended with high transparency (*intensity * 0.38*), ensuring that our dark *#1C1917* headlines consistently pass WCAG AAA contrast."

4. **Performance & Access:**

- "We cap DPR at 2 so 4K laptops don't heat up, stop rendering when the tab is inactive, and respect reduced motion preferences."